4.14.3.4 Example technical skills

4.14.3.4.1 Table 1: Example technical skills

Group	Model (including data model/structure)	Algorithms
A	Complex data model in database (eg several interlinked tables)	Cross-table parameterised SQL Aggregate SQL functions User/CASE-generated DDL script
	Hash tables, lists, stacks, queues, graphs, trees or structures of equivalent standard Files(s) organised for direct access	Graph/Tree Traversal List operations Linked list maintenance Stack/Queue Operations Hashing
	Complex scientific/mathematical/robotics/control/business model	Advanced matrix operations Recursive algorithms Complex user-defined algorithms (eg optimisation, minimisation, scheduling, pattern matching) or equivalent difficulty Mergesort or similarly efficient sort
	Complex user-defined use of object-orientated programming (OOP) model, eg classes, inheritance, composition, polymorphism, interfaces Complex client-server model	Dynamic generation of objects based on complex user-defined use of OOP model Server-side scripting using request and response objects and server-side extensions for a complex client-server model Calling parameterised Web service APIs and parsing JSON/XML to service a complex client-server model

Group	Model (including data model/structure)	Algorithms
B	Simple data model in database (eg two or three interlinked tables) Multi-dimensional arrays Dictionaries Records Text files File(s) organised for sequential access Simple scientific/mathematical /robotics/ control/business model Simple OOP model Simple client-server model	Single table or non-parameterised SQL Bubble sort Binary search Writing and reading from files Simple user defined algorithms (eg a range of mathematical/statistical calculations) Generation of objects based on simple OOP model Server-side scripting using request and response objects and server-side extensions for a simple client-server model Calling Web service APIs and parsing JSON/ XML to service a simple client-server model
C	Single-dimensional arrays Appropriate choice of simple data types Single table database	Linear search Simple mathematical calculations (eg average) Non-SQL table access

Note that the contents of Table 1 are examples, selected to illustrate the level of demand of the technical skills that would be expected to be demonstrated in each group. The use of alternative algorithms and data models is encouraged. If a project cannot easily be marked against Table 1 (for example, a project with a considerable hardware component) then please consult your AQA non-exam assessment adviser or provide a full explanation of how you have arrived at the mark for this section when submitting work for moderation.

4.14.3.4.2 Table 2: Coding styles

Style	Characteristic
Excellent	Modules (subroutines) with appropriate interfaces Loosely coupled modules (subroutines) – module code interacts with other parts of the program through its interface only Cohesive modules (subroutines) – module code does just one thing Modules(collections of subroutines) – subroutines with common purpose grouped Defensive programming Good exception handling
Good	Well-designed user interface Modularisation of code Good use of local variables Minimal use of global variables Managed casting of types Use of constants Appropriate indentation Self-documenting code Consistent style throughout File paths parameterised
Basic	Meaningful identifier names Annotation used effectively where required

The descriptions in Table 2 are cumulative, ie for a program to be classified as excellent it would be expected to exhibit the characteristics listed as excellent, good and basic not just those listed as excellent.